

DOCUMENTO TÉCNICO DE DISEÑO

Ecommify

Arquitectura Polígloa Híbrida

PostgreSQL + MongoDB

Diseño y Optimización de Bases de Datos

Luis Miguel Ossa Arias

Javier Ricardo Sandoval

Jose Luis Mora Serrano

Maestría en Arquitectura de Software · Facultad de Ingeniería · Universidad de La Sabana

Docente: Miguel Alfonso Varela Fonseca · 22 de mayo de 2026

Contexto

Marketplace brasileño · Dataset público de Olist (Kaggle)

¿Qué es Ecommify?

Plataforma e-commerce intermediaria entre vendedores independientes y compradores, que gestiona el ciclo completo del pedido: compra → aprobación → envío → entrega → reseña post-servicio.

Naturaleza heterogénea del dato

- ▶ **Datos transaccionales** (pedidos, pagos, vínculos cliente-vendedor): altamente estructurados, requieren ACID e integridad referencial.
- ▶ **Catálogo de productos**: atributos variables por categoría — un electrónico requiere voltaje y garantía; una prenda, talla y material. Ideal para modelo de documento.

DECISIÓN ARQUITECTÓNICA

Polyglot Persistence

Cada motor gestiona los datos para los que está técnicamente optimizado, evitando forzar todo a un único modelo.

PostgreSQL

Módulo transaccional

- ACID estricto
- Integridad referencial
- Esquema 3FN
- OLTP intensivo

MongoDB

Módulo NoSQL

- Esquema flexible
- Catálogo enriquecido
- Logs y eventos
- Sesiones TTL

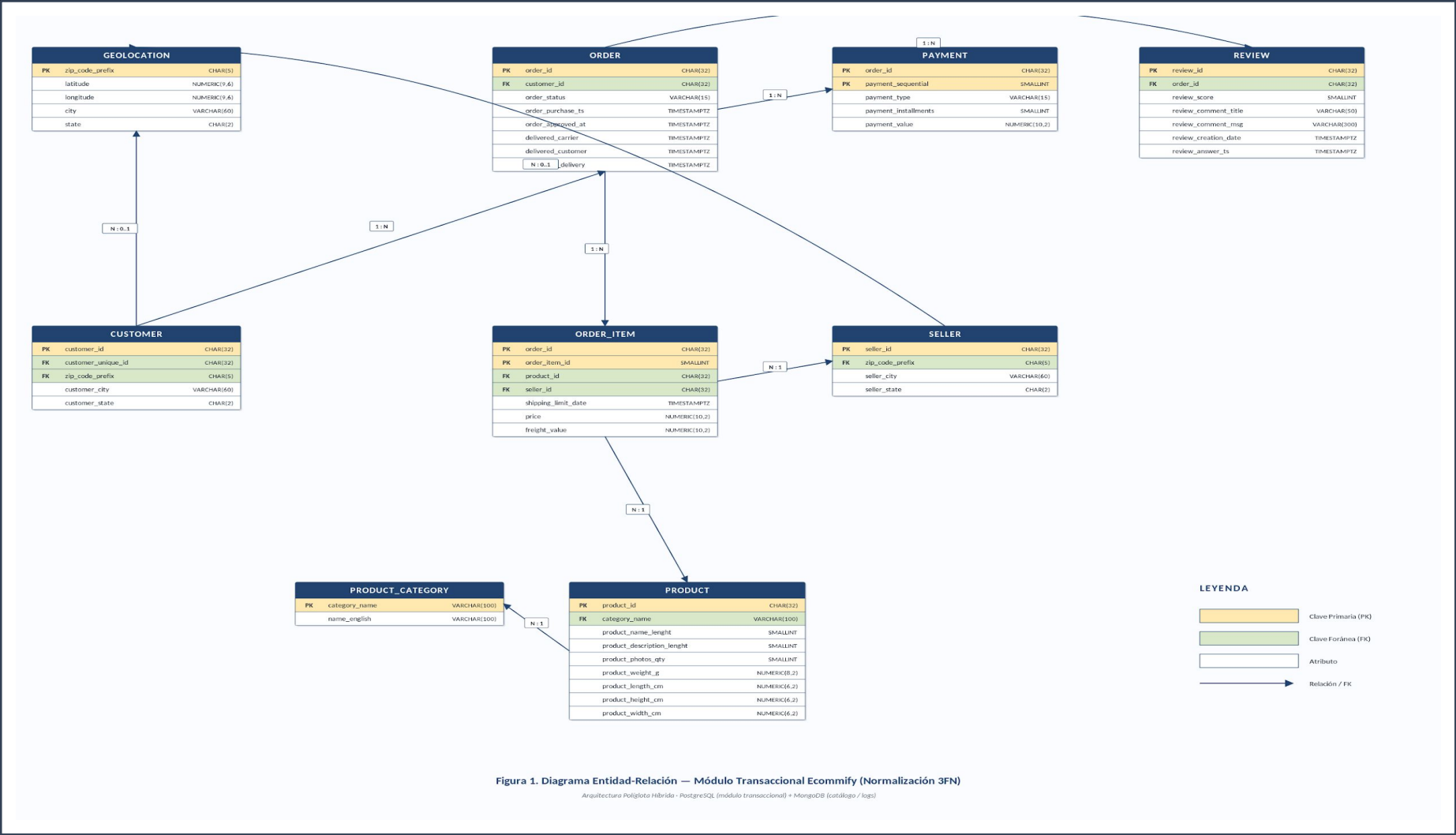
Requisitos del Sistema

Funcionales (qué hace) y No Funcionales (cómo lo hace)

REQUISITOS FUNCIONALES		REQUISITOS NO FUNCIONALES	
RF1	Registro y autenticación de clientes customer_id (sesión) + customer_unique_id (identidad real)	RNF1	Consistencia e integridad referencial ACID en PostgreSQL — sin ítems huérfanos
RF2	Gestión del catálogo de productos Categorías PT/EN, atributos variables por categoría (MongoDB)	RNF2	Escalabilidad de datos estáticos 1M+ registros geolocation con JOIN eficiente (B-Tree)
RF3	Procesamiento de pedidos con ciclo de vida 8 estados: created → approved → invoiced → ... → delivered	RNF3	Flexibilidad del catálogo Atributos por categoría sin ALTER TABLE (MongoDB)
RF4	Gestión de ítems por pedido Múltiples productos de distintos vendedores con flete por ítem	RNF4	Rendimiento en consultas frecuentes Índices en columnas de alta cardinalidad
RF5	Procesamiento de pagos múltiples credit_card, debit_card, boleto, voucher · soporte cuotas	RNF5	Normalización hasta 3FN Sin dependencias parciales ni transitivas
RF6	Gestión de vendedores con geolocalización Análisis de proximidad cliente-vendedor	RNF6	Tipos de datos precisos NUMERIC (dinero), TIMESTAMPTZ (multi-zona), CHAR(32) (hashes)
RF7	Sistema de reseñas post-entrega Likert 1-5 + título + cuerpo con timestamps	RNF7	Extensibilidad arquitectónica Nuevos módulos (logs, ML) sin afectar core transaccional
RF8	Geolocalización por código postal Resolución por ZIP prefix de 5 dígitos		

Diagrama Entidad-Relación

9 entidades normalizadas hasta 3FN · Módulo Transaccional



9 ENTIDADES

CUSTOMER	99,441
ORDER	99,441
ORDER_ITEM	112,650
PAYMENT	103,886
REVIEW	99,224
PRODUCT	32,951
PRODUCT_CATEGORY	71
SELLER	3,095
GEOLOCATION	1,000,163

Clave de sincronización:

product_id CHAR(32)

Hash MD5 compartido entre PostgreSQL y MongoDB

Esquema Relacional Normalizado (3FN)

Violaciones detectadas en OrdersFull → entidades independientes

Entidad	Violación FN	Problema en OrdersFull	Solución 3FN
ORDER	1FN	Datos del pedido repetidos en cada fila de ítem	Entidad ORDER separada con order_id PK
ORDER_ITEM	2FN	price/freight dependían del ítem completo, no de PK compuesta	PK (order_id, order_item_id); FK → PRODUCT, SELLER
PRODUCT	2FN	peso, dimensiones dependían solo de product_id	Tabla PRODUCT con atributos físicos propios
PAYMENT	3FN	payment_type literal repetido + múltiples pagos por orden	Tabla independiente con PK compuesta (order_id, sequential)
REVIEW	3FN	review_score mezclado con datos de ítem	Tabla REVIEW vinculada a ORDER (CASCADE)
PRODUCT_CATEGORY	3FN	name_english dependía de category_name, no de product_id	Catálogo de categorías con clave natural (71 valores)
CUSTOMER	3FN	city/state dependían de zip_code_prefix (transitiva)	FK customer.zip_code_prefix → GEOLOCATION
SELLER	3FN	Mismo patrón que CUSTOMER	FK seller.zip_code_prefix → GEOLOCATION
GEOLOCATION	1FN	Múltiples filas por ZIP (hasta 141 coords por código)	Promedio ponderado de lat/lng — 1 fila por ZIP

► 9 entidades · 9 violaciones de forma normal resueltas · 0 anomalías de inserción, actualización o borrado.

Tipos Avanzados en PostgreSQL

5 tipos especializados que reducen JOINS y habilitan operadores nativos

Tipo	Campo	Índice	Operadores	Justificación
JSONB	product.specifications	GIN	->, ->>, @>	7 columnas planas → 1 documento extensible por categoría. Sin ALTER TABLE al agregar nuevas specs.
TEXT[]	product.photo_urls	GIN	array_length, ANY	URLs del CDN como array. Evita tabla photos separada con JOIN adicional.
TSTZRANGE	orders.promotion_period	GIST	@>, &&	Período activo de la orden. Consultas de solapamiento sin rango manual entre 2 columnas.
ENUM	orders.order_status	B-Tree	=, IN, ORDER BY	Dominio cerrado de 8 valores. Validación automática + almacenamiento como entero.
POINT	customer/seller (view)	GIST	<-> (distancia)	Lat/Lng como punto inseparable. Operador <-> para proximidad sin extensión PostGIS.

BENEFICIO: Reducción de JOINS · operadores indexados nativos · consultas más simples sin lógica en capa de aplicación.

Extensiones de PostgreSQL

Evaluadas para Supabase Free Tier · 2 adoptadas, 1 diferida, 1 descartada

 ADOPTAR pg_trgm CASO DE USO Búsqueda tolerante a errores tipográficos en nombres de productos y categorías. JUSTIFICACIÓN Habilitada por defecto en Supabase. Crea índices GIN/GIST sobre trigramas. Permite similarity() > 0.3 sin dependencia externa.	 ADOPTAR pgcrypto CASO DE USO Generación y validación de hashes MD5 nativamente en la base de datos. JUSTIFICACIÓN Disponible en Supabase. gen_random_uuid() y digest() sin depender de capa de aplicación para generar IDs.	 DIFERIR PostGIS CASO DE USO Optimización de costos de envío basados en distancia geoespacial vendedor-cliente. JUSTIFICACIÓN No disponible en Supabase Free Tier. El tipo POINT con operador <-> cubre las consultas básicas. Incorporar en fase Pro.	 DESCARTAR hstore CASO DE USO Alternativa a JSONB para almacenamiento de pares clave-valor. JUSTIFICACIÓN JSONB supera a hstore en todas las dimensiones: operadores más ricos, soporte de tipos anidados, índices GIN más eficientes.
--	--	--	--

Esquema MongoDB

3 colecciones · documentos JSON/BSON · esquema flexible y patrones especializados

products

~33K docs · +20% anual

PATRÓN: Extended Reference + Subset + Schema Versioning

Catálogo enriquecido con specifications variables por categoría. product_id como clave de sincronización con PostgreSQL.

event_logs

Alto volumen · +50% anual

PATRÓN: Bucket Pattern + TTL (90 días)

Vistas de producto, clics, búsquedas, carritos abandonados. Esquema heterogéneo intencional.

user_sessions

Variable · TTL 24h

PATRÓN: TTL Index automático

Carritos temporales y sesiones activas. MongoDB elimina automáticamente al expirar (sin INSERT/DELETE relacional).

EJEMPLO · documento de products

```
{
  "_id": "ObjectId(' ... ')",
  "product_id": "ffe9c82b9f56afc0dfe57d81de5f5a5",
  "category_name": "informatica_acessorios",
  "category_name_english": "computers_accessories",
  "name": "[Anonimizado por Olist]",
  "description": "Descripción completa del producto ... ",
  "photo_urls": [
    "https://cdn.ecommify.com/products/img1.jpg",
    "https://cdn.ecommify.com/products/img2.jpg"
  ],
  "specifications": {
    "weight_g": 300,
    "dimensions_cm": { "length": 20, "height": 5, "width": 15 },
    "connectivity": ["USB-A", "Bluetooth 5.0"],
    "compatibility": ["Windows", "macOS", "Linux"],
    "warranty_months": 12
  },
  "tags": ["periferico", "mouse", "inalambrico"],
  "avg_review_score": 4.3,
  "total_reviews": 128,
  "created_at": "ISODate('2024-01-15T10:00:00Z')",
  "updated_at": "ISODate('2024-06-20T14:30:00Z')",
}
```

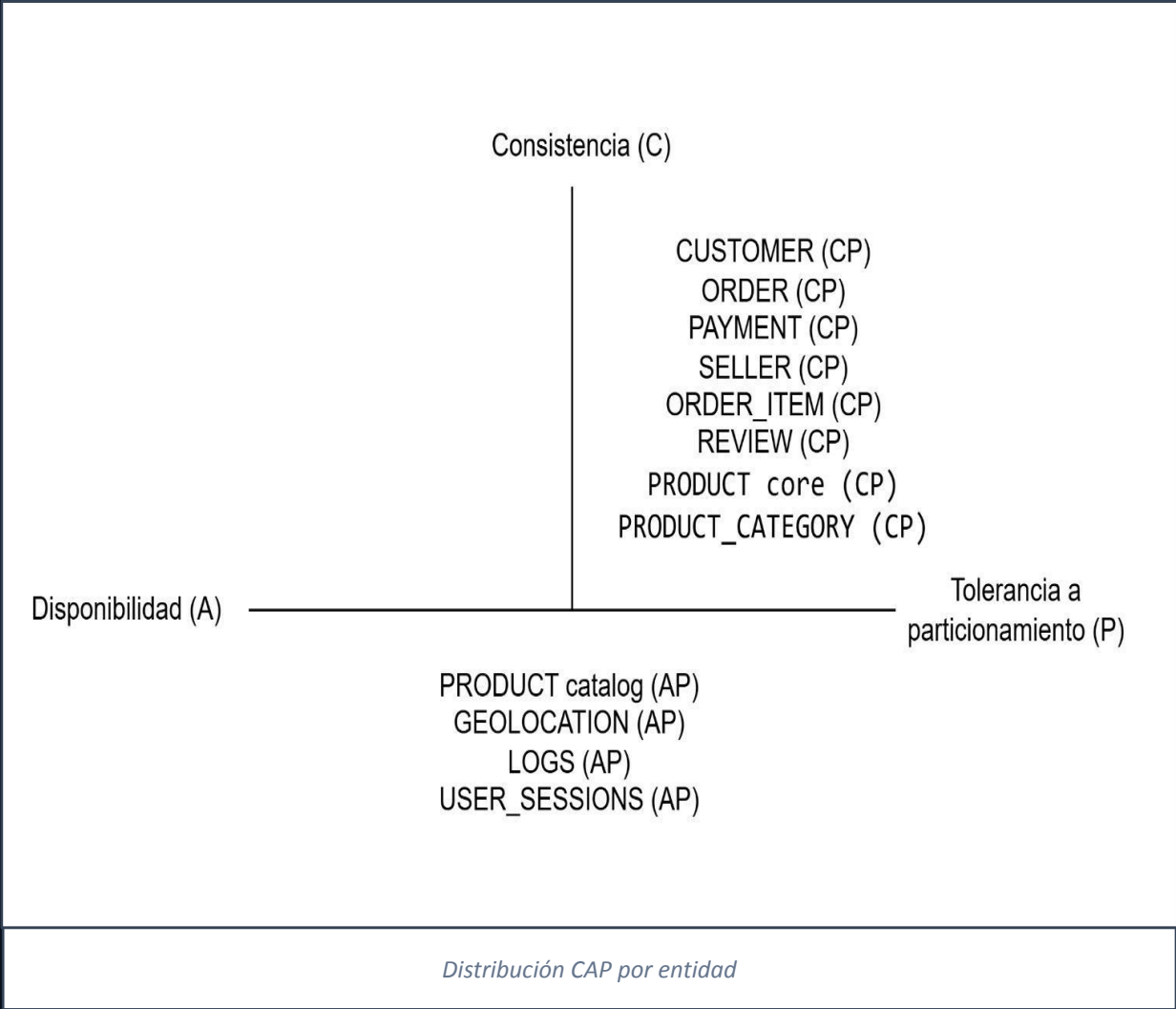

Matriz de Decisión por Entidad

Asignación PostgreSQL vs MongoDB · criterios CAP + naturaleza + patrón de acceso

Módulo / Colección	CAP	Naturaleza	Patrón Acceso	Motor	Justificación
CUSTOMER	CP	Estructurada	Transaccional	PostgreSQL	Datos personales sensibles. ACID + integridad FK.
ORDER	CP	Estructurada	OLTP intensivo	PostgreSQL	Núcleo transaccional. Evita pedidos duplicados.
ORDER_ITEM	CP	Estructurada	OLTP / OLAP	PostgreSQL	Entidad débil con PK compuesta. FK a ORDER/PROD/SELLER.
PAYMENT	CP	Estructurada	OLTP crítico	PostgreSQL	Operaciones financieras. Cero tolerancia a inconsistencia.
REVIEW	CP	Estructurada	OLAP / escritura	PostgreSQL	Atributos estables. CASCADE con ORDER.
PRODUCT (core)	CP	Estructurada	Transaccional	PostgreSQL	Métricas físicas (peso, dim). FK a categoría.
PRODUCT_CATEGORY	CP	Estructurada	Catálogo estático	PostgreSQL	Clave natural estable. Resuelve dependencia 3FN.
SELLER	CP	Estructurada	OLTP moderado	PostgreSQL	Datos fiscales y contractuales.
GEOLOCATION	AP	Semi-estruct.	Lectura intensiva	PostgreSQL	1M registros. Índice B-Tree suficiente.
products (catálogo)	AP	Semi-estruct.	Lectura intensiva	MongoDB	Atributos variables por categoría. Esquema flexible.
event_logs	AP	Semi-estruct.	Escritura masiva	MongoDB	TTL natural. Alta disponibilidad > consistencia.
user_sessions	AP	Semi-estruct.	Escritura/Lectura	MongoDB	Documentos efímeros con TTL 24h.

Análisis del Teorema CAP

Tolerancia a particiones inevitable → decisión real: CP vs AP por entidad



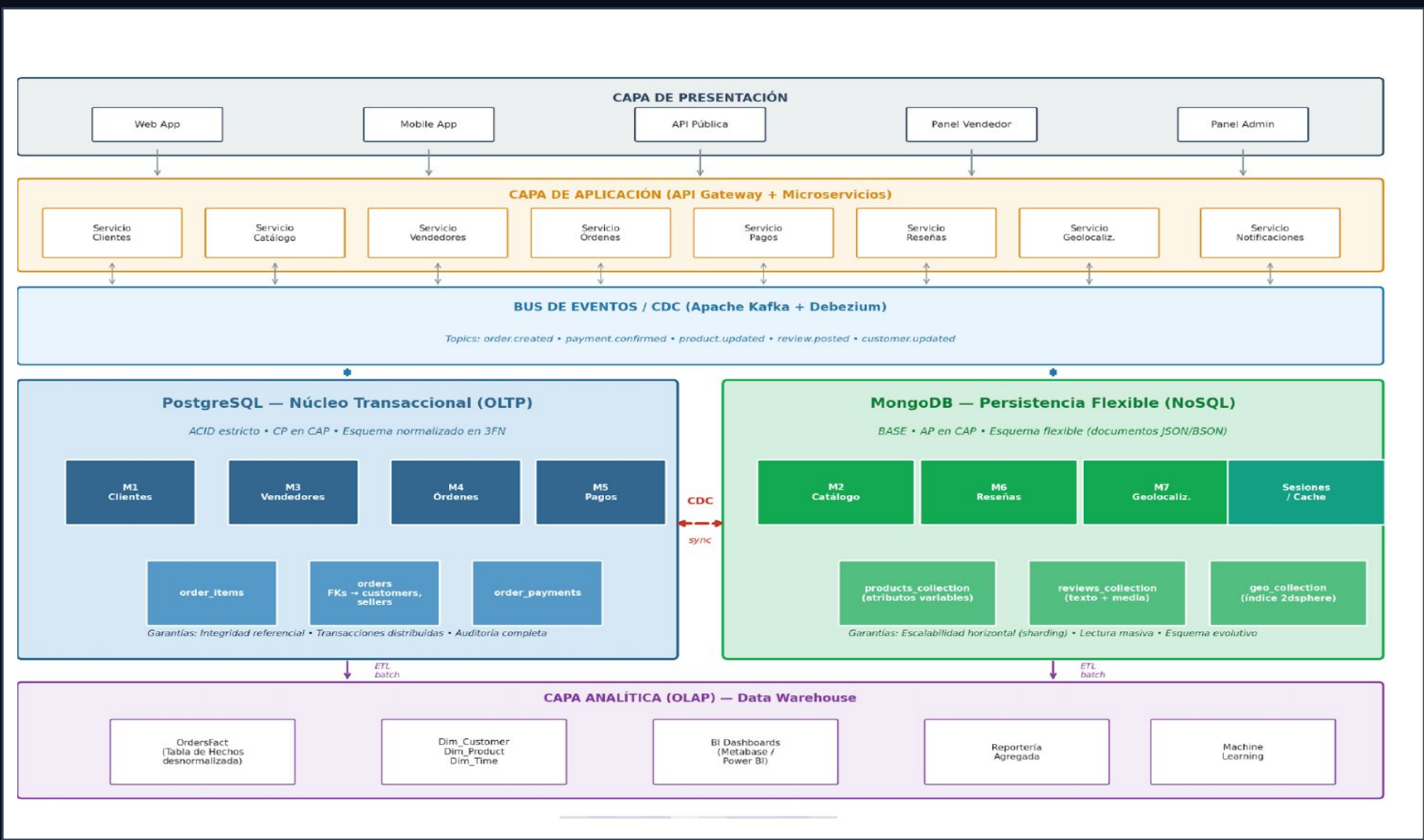
CP · Consistencia > Disponibilidad	
CUSTOMER	Datos personales sensibles · pérdida de identidad inaceptable
ORDER	Pedido duplicado → pérdida financiera y legal
ORDER_ITEM	Integridad referencial · ítem huérfano es ilegítimo
PAYMENT	Operaciones financieras · double-spending inaceptable
REVIEW	Una reseña debe corresponder exactamente a una orden
SELLER	Datos contractuales requieren consistencia
PRODUCT (core)	Métricas físicas estables · FK desde ORDER_ITEM exige integridad
PRODUCT_CATEGORY	Clave natural estable · resuelve dependencia transitiva 3FN

AP · Disponibilidad > Consistencia	
PRODUCT catalog	Alta disponibilidad para consultas · consistencia eventual tolerable
GEOLOCATION	Consultas de proximidad · disponibilidad > consistencia estricta
LOGS / events	Escritura masiva · si se pierde un evento, el sistema no falla
USER SESSIONS	Datos efímeros con TTL 24h · expiración automática sin coordinación

► 8 entidades CP en PostgreSQL · 4 entidades AP en MongoDB. La arquitectura híbrida resuelve el dilema sin compromisos.

Arquitectura Híbrida Propuesta

5 capas · CDC con Debezium + Kafka · Outbox Pattern para eventos críticos



Flujo A · CDC

Change Data Capture

1. INSERT/UPDATE/DELETE → WAL
2. Debezium lee WAL → Kafka
3. Consumidores → MongoDB / DW
4. Latencia < 1 segundo

Flujo B · Outbox

Eventos críticos atómicos

1. ACID start en PostgreSQL
2. Write cambio + event_outbox
3. Relay publica en Kafka
4. Reintento o rollback total

Limitaciones y Mitigaciones

Trade-offs arquitectónicos identificados con sus estrategias de mitigación

Trade-off	Decisión	Riesgo	Mitigación
Normalización 3FN vs rendimiento de consultas	<i>Normalizar todo el módulo transaccional a 3FN</i>	JOINS múltiples impactan consultas analíticas	Índices compuestos (B-Tree, GIN) · Vistas materializadas para reportes · Particionamiento por fecha en ORDER
Persistencia políglota vs complejidad operativa	<i>PostgreSQL + MongoDB</i>	Dos motores que administrar, sincronización entre ellos	CDC con Debezium + Kafka para replicación asíncrona · Outbox Pattern para eventos críticos · product_id como clave compartida
Tipos avanzados PostgreSQL	<i>JSONB, TEXT[], TSTZRANGE, ENUM, POINT</i>	Curva de aprendizaje · acoplamiento a features específicas	Todos son parte del estándar PostgreSQL ≥ 12 · Documentación extensa · Reducción de JOINS y consultas más simples
Desnormalización controlada	<i>Mantener city/state en CUSTOMER y SELLER pese a FK</i>	Datos redundantes · posible inconsistencia si cambia GEOLOCATION	Redundancia de conveniencia (dato ya presente al insertar) · ON UPDATE CASCADE en FK propaga cambios de ZIP
Particionamiento de ORDER	<i>RANGE PARTITIONING por order_purchase_timestamp</i>	Complejidad en migraciones y mantenimiento de particiones	Partition pruning automático · Tabla order_future para datos fuera de rango · Evaluar reparticionado a los 5M filas
Múltiples pagos por orden	<i>PK compuesta (order_id, payment_sequential)</i>	Consultas de agregación requieren SUM + GROUP BY	Indexar order_id en PAYMENT · Vista materializada v_order_total_paid

Plan de Implementación

4 fases progresivas · proyecto académico Maestría Arquitectura de Software

01 Diseño & Modelado Semana 1-2 ACTIVIDADES ▪ Análisis del dataset Olist (9 CSVs) ▪ Identificación de violaciones FN ▪ Diseño ER + normalización 3FN ▪ Definición de tipos avanzados ENTREGABLE Documento técnico · Diagrama ER	02 Implementación PostgreSQL Semana 3-4 ACTIVIDADES ▪ DDL completo en Supabase ▪ Carga del dataset (seed data) ▪ Triggers y vistas materializadas ▪ Estrategia de indexación ENTREGABLE Script SQL · DB poblada · Consultas	03 Módulo MongoDB Semana 5-6 ACTIVIDADES ▪ Diseño de 3 colecciones ▪ Carga del catálogo enriquecido ▪ Configuración TTL Indexes ▪ Validación de patrones aplicados ENTREGABLE Esquemas JSON · Colecciones pobladas	04 Integración & Sustentación Semana 7-8 ACTIVIDADES ▪ Sincronización PostgreSQL ↔ MongoDB ▪ Consultas analíticas demo ▪ Validación de requisitos ▪ Preparación de presentación ENTREGABLE Demo funcional · Defensa oral
--	--	---	---

¿Por qué arquitectura políglota híbrida?

1

Integridad sin compromiso

PostgreSQL garantiza ACID, integridad referencial y consistencia para datos críticos (órdenes, pagos, clientes) mediante normalización 3FN, FK con RESTRICT/CASCADE y dominios cerrados (ENUM, CHECK).

2

Flexibilidad sin sacrificio

MongoDB absorbe la heterogeneidad del catálogo y los logs operacionales con esquema variable, patrones Extended Reference, Bucket Pattern y TTL automático, sin migraciones ALTER TABLE.

3

Escalabilidad independiente

Cada motor escala según su naturaleza. CDC con Debezium + Kafka mantiene coherencia entre fuentes con latencia < 1s sin acoplar las cargas OLTP y NoSQL.

Polyglot Persistence · *cada motor para los datos que está optimizado a gestionar.*